Views

When to use this

Views are ideal for multi-step workflows—dashboards, menus, or game scenes—where you want to encapsulate options and state per screen. They integrate prompts with lifecycle events so you can build entire applications from composable classes.

Key types

- `StaticView` – Presents a fixed set of options defined via attributes or collection initialisation.
- `DynamicView` – Rebuilds its option list every time it renders, enabling state-aware labels and colours.
- `ViewOption` / `DynamicOption<T>` – Describe the work performed when a user selects an option.
- `ChoiceCallback` – Optional delegate that receives notification whenever an option is executed.

Minimal example

```
[View("Main menu")]
public class MainMenu : StaticView
{
    public MainMenu() : base()
    {
    }

    [ViewOption("Start job", ConsoleColor.Green)]
    private void StartJob()
    {
        Consoul.Write("Starting job...", ConsoleColor.Green);
        GoBack();
    }

    [ViewOption("Show high scores", ConsoleColor.Cyan)]
    private void ShowScores()
    {
        Consoul.Write("Loading scores...", ConsoleColor.Yellow);
        NavigateTo<HighScoresView>(replace: false);
    }

    [ViewOption("Exit", ConsoleColor.DarkGray)]
    private void Exit()
    {
```

```
        Consoul.Write("Goodbye!", ConsoleColor.Gray);
        GoBack();
    }
}
```

Navigation helpers

`StaticView` and `DynamicView<T>` expose protected helpers to request navigation without manually instantiating other views. Call `NavigateTo<TView>()` to replace the current view (the default), `NavigateTo<TView>(replace: false)` to push a new view on the stack, `NavigateTo(() => new DetailView(arg))` when you need to supply constructor parameters, or `GoBack()` to pop to the previous screen. The renderer executes these requests inside an iterative loop, preventing recursive render chains and the stack overflows that could occur when options previously invoked `new OtherView().Render()`.

Advanced tips

- **Attribute-driven options** – Decorate methods with `ViewOptionAttribute` or `DynamicViewOptionAttribute` to declare choices alongside their logic. Attribute metadata can control colour, ordering, and whether an option is hidden until a predicate returns true; leverage these hooks to gate admin-only commands or feature flags.
- **Go back behaviour** – Each view automatically appends a "Go back" option and now uses `PromptResult` cancelation rather than the legacy `Consoul.EscapeIndex` constant. Customise the label via `ViewAttribute.GoBackMessage` or by changing `RenderOptions.DefaultGoBackMessage`, and check `GoBackRequested` after `RenderAsync` returns to run cleanup when a user leaves mid-operation.
- **Async rendering** – Use `RenderAsync()` on dynamic views when options trigger asynchronous operations; the framework awaits each action before re-rendering. Combine this with `CancellationToken` parameters on option handlers to respond to user aborts or window closures, and surface progress updates by re-rendering interim messages via `Consoul.Write`.
- **Shared state & navigation** – Pass dependencies via constructors or properties; views are regular classes and can store fields, timers, or other stateful components. Use `NavigateTo<TView>()` (or `NavigateTo<TView>(replace: false)` when you need to push) or `GoBack()` inside an option to request transitions; the renderer now applies these commands iteratively so the call stack stays flat even through long navigation chains. Inject shared services (logging, data providers) to keep options thin and testable.
- **Composed dashboards** – `DynamicView` options can render tables or progress bars inline, then rehydrate their options list after completing background work. This makes it easy to build monitoring dashboards driven by timers or file watchers; schedule refreshes with `System.Threading.Timer`

to trigger `RenderAsync()` when data changes.

- **Choice callbacks** – Supply a `ChoiceCallback` to track navigation analytics, enforce authorisation, or log audit trails. Because the callback receives the executed option, you can centralise cross-cutting concerns instead of duplicating logic inside every handler.

Object editing

`EditObjectView` now renders the target model as annotated JSON so users can arrow through property values before pressing Enter to edit. Each line includes JavaScript-style comments sourced from `[Display]` metadata and the model's XML documentation (including `<see cref="..."/>` references). Use the left/right or up/down arrows—or the Tab key—to change the selection, and Esc to return to the menu.

Metadata-aware prompts

When a property is selected the editor surfaces the resolved display name, description and summary before invoking an appropriate prompt. For simple types the default prompt respects the property's type; string properties ending with `Path` automatically use `Consoul.PromptForFilepath`. You can opt into other editors by decorating properties with `PropertyEditorAttribute`:

```
public sealed class AdapterConfiguration
{
    [PropertyEditor(typeof(FilePathPropertyEditor))]
    [Display(Name = "Adapter", Description = "Full path to the adapter implementation.")]
    public string AdapterPath { get; set; } = string.Empty;
}
```

If you need to normalise values before assignment—for example trimming a partial path—implement `IPropertyValueFormatter` and apply `PropertyValueFormatterAttribute` to the property. Formatters receive the edit context so they can inspect the model, documentation and original value:

```
public sealed class RelativePathFormatter : IPropertyValueFormatter
{
    public object Format(PropertyEditContext context, object value)
    {
        if (value is string text)
        {
            return Path.GetRelativePath(AppContext.BaseDirectory, text);
        }

        return value;
    }
}
```

```
public sealed class ScriptOptions
{
    [PropertyEditor(typeof(FilePathPropertyEditor))]
    [PropertyValueFormatter(typeof(RelativePathFormatter))]
    public string ScriptPath { get; set; } = string.Empty;
}
```

Complex types (objects, collections and dictionaries) still launch nested `EditObjectView` instances so you can drill into hierarchies while preserving the legacy menu options for power users.

Metadata resolvers

Some properties surface values whose structure is only known at runtime—for example, dictionaries that map to constructor parameters in an external assembly. Decorate these members with `PropertyMetadataResolverAttribute` to plug in a custom resolver that supplies documentation, editors or formatters without relying on reflection against the runtime value.

```
public sealed class OptionsMetadataResolver : IPropertyMetadataResolver
{
    public ResolvedPropertyMetadata Resolve(PropertyInfo property)
    {
        var documentation = new PropertyDocumentation(
            property,
            "Options",
            "Key/value pairs configuring the remote adapter.",
            () => "Each key should match a constructor parameter on the adapter type.");

        return new ResolvedPropertyMetadata(
            documentation,
            new DefaultPropertyEditor(),
            null);
    }
}

public sealed class AdapterConfiguration
{
    [PropertyMetadataResolver(typeof(OptionsMetadataResolver))]
    public Dictionary<string, object> Options { get; set; } = new Dictionary<string, object>
}
```

Resolvers can construct rich guidance even when the property value references remote types, enabling consistent prompts and comments throughout the JSON editor.

Layered editors Some data surfaces as opaque `object` graphs—think JSON blobs, plugin settings loaded from another assembly, or dictionaries whose values describe constructor parameters. To keep these scenarios discoverable the resolver can also expose additional layers through `IPropertyLayerProvider`. Each layer describes a named editing surface with an optional description and a handler that receives the active `PropertyEditContext`. When the user presses Enter on the property, Consoul prompts whether to run the default editor or one of the custom layers.

Implementations can parse JSON Schema files, spin up another `EditObjectView`, or invoke bespoke editors for nested structures. The handler can mutate `context.CurrentValue` and return `true` to signal that the new value should be applied automatically. If the layer wants to persist the value directly it can call `context.ApplyValue` and still return `true` while marking the `PropertyEditLayer` as not applying the context value. Returning `false` now falls back to the default editor so layers can bootstrap data (for example by materialising dictionary entries) and then let the built-in prompts continue the session.

```
public sealed class OptionsMetadataResolver : IPropertyMetadataResolver
{
    public ResolvedPropertyMetadata Resolve(PropertyInfo property)
    {
        var documentation = new PropertyDocumentation(
            property,
            "Options",
            "Key/value pairs configuring the remote adapter.",
            () => "Each key should match a constructor parameter on the adapter type.");

        return new ResolvedPropertyMetadata(
            documentation,
            new DefaultPropertyEditor(),
            null,
            new SchemaLayerProvider());
    }
}

private sealed class SchemaLayerProvider : IPropertyLayerProvider
{
    public IEnumerable<PropertyEditLayer> GetLayers(PropertyEditContext context)
    {
        return new[]
        {
            new PropertyEditLayer(
                "Edit via schema",
                "Loads the remote JSON schema and opens a nested editor for the selected dic
                RunSchemaEditor,
```

```
                true)
        };
    }

    private static bool RunSchemaEditor(PropertyEditContext context)
    {
        // Locate the JSON schema based on the current model and open a tailored view.
        var view = new EditObjectView(JsonSchemaParser.BuildModel(context.CurrentValue));
        view.Render();
        context.CurrentValue = view.Model;
        return true; // the updated model will be applied by Consoul.
    }
}
```

Because layers participate in the same prompt flow, users can switch between schema-driven editors, remote type reflection and the built-in prompts without leaving the JSON workspace.